

1. Create an account in AWS and configure a budget

Amazon Web Services (AWS) is a cloud computing platform that provides services like storage, networking, databases, and virtual servers. Creating an AWS account is the first step to accessing these cloud services. After account creation, configuring a budget is important to monitor cloud expenses and avoid unexpected charges. AWS Budgets allows users to set spending limits and receive alerts when usage exceeds the defined threshold. This helps organizations and students manage cloud costs efficiently and maintain financial control over cloud resources.

2. Create MFA for authentication

Multi-Factor Authentication (MFA) is a security feature used to provide additional protection to AWS accounts. In MFA, users must provide two forms of authentication, such as a password and a verification code from a mobile application. This reduces the risk of unauthorized access even if the password is compromised. AWS supports MFA devices like Google Authenticator and hardware tokens. Enabling MFA improves account security and is considered a best practice in cloud computing environments.

3. Create IAM user and give full access to S3

AWS Identity and Access Management (IAM) is a service used to securely control access to AWS resources. An IAM user is created to provide limited or specific access to users instead of using the root account. By assigning full access to Amazon S3, the user can create, upload, delete, and manage files in S3 buckets. IAM policies are attached to define permissions. This improves security and allows proper management of cloud resources in an organization.

4. Create a private bucket in AWS

Amazon S3 bucket is a cloud storage container used to store files and objects. A private bucket restricts access so that only authorized users can view or download the files. After uploading a file, a pre-signed URL can be generated to temporarily allow access to the private object. This method ensures secure sharing of sensitive data for a limited time. Private buckets are widely used in organizations to maintain confidentiality and data protection.

5. Create a public bucket in AWS

A public S3 bucket allows files to be accessed by anyone using the file URL. After creating the bucket, proper bucket policies and permissions are configured to make objects publicly accessible. Files such as images, documents, or website resources can then be shared over the internet. Public buckets are commonly used for hosting public content, but security precautions must be taken to prevent unauthorized modification or deletion of files.

6. Upload a static website on S3

Amazon S3 can be used to host static websites containing HTML, CSS, JavaScript, and media files. Static website hosting allows users to make web pages accessible over the internet without using a web server. After enabling static website hosting and uploading website files, AWS provides a website endpoint URL for access. This method is cost-effective, scalable, and suitable for portfolio websites, landing pages, and small web applications.

7. Hosting a website on EC2

Amazon EC2 (Elastic Compute Cloud) provides virtual servers in the cloud. A website can be hosted on EC2 by launching an instance, installing a web server such as Apache or Nginx, and uploading website files. Security groups are configured to allow HTTP or HTTPS traffic. EC2 hosting provides more flexibility and control compared to static hosting because dynamic applications and databases can also be managed on the server.

8. Deploy a project from a local machine to GitHub and vice versa

GitHub is a platform used for version control and project management using Git. A project can be uploaded from a local machine to GitHub using Git commands like `git init`, `git add`, `git commit`, and `git push`. Similarly, projects can be downloaded from GitHub to a local machine using `git clone` or updated using `git pull`. This process helps developers maintain project versions, collaborate efficiently, and store code securely online.

9. Deploy a project from GitHub to EC2

A project stored on GitHub can be deployed to an EC2 instance by connecting the instance through SSH and cloning the repository using Git. Required software and dependencies are installed on the server, and the application is configured to run properly. This process helps automate deployment and ensures that the latest version of the project can be easily accessed and updated on the cloud server.

10. Deploy a project from GitHub to EC2 using security group and user data

In AWS EC2, security groups act as virtual firewalls that control incoming and outgoing traffic. User data scripts are used to automate server configuration during instance launch. By creating a security group and adding user data commands, the EC2 instance can automatically install required software, clone the GitHub project, and start the application. This reduces manual work and speeds up deployment.

11. Build scaling plans in AWS

AWS Auto Scaling helps automatically increase or decrease the number of EC2 instances based on traffic and workload. Load balancing distributes incoming traffic across multiple instances to improve availability and performance. Scaling plans ensure applications remain responsive during high traffic while reducing costs during low usage periods. This improves reliability, fault tolerance, and efficient resource utilization in cloud environments.

12. Deploy and run the project in AWS without using the port

Projects can be deployed in AWS without directly exposing application ports by using reverse proxy services such as Nginx or Apache. The reverse proxy forwards requests from standard ports like 80 or 443 to the application running internally. This improves security and allows easier public access through domain names or load balancers. It also helps manage HTTPS connections and multiple applications on a single server.

13. Create a WorkMail for organization

Amazon WorkMail is a secure business email and calendar service provided by AWS. It allows organizations to create professional email accounts with custom domains. WorkMail supports secure communication, calendar sharing, and integration with desktop and mobile email applications. It helps organizations manage communication efficiently while maintaining security and reliability in cloud-based email services.

14. Create an Elastic IP for an instance

An Elastic IP is a static public IP address assigned to an EC2 instance. Normally, EC2 public IP addresses may change when the instance restarts. Elastic IP solves this problem by providing a permanent IP address.

Elastic IPs are useful for hosting websites, applications, and DNS mapping because users can always access services using the same IP address. If an instance fails, the Elastic IP can quickly be reassigned to another instance. This improves availability and reduces downtime in production environments.

15. Create a Serverless computing service

Serverless computing allows applications to run without managing servers. AWS Lambda is a popular serverless service where code executes automatically in response to events. Developers only upload the code, and AWS manages scaling, infrastructure, and availability. Serverless architecture reduces operational overhead, improves scalability, and charges users only for the execution time used.

16. Manage Amazon DNS service and run a project using domain name URL

Amazon Route 53 is a scalable DNS service used to manage domain names and route traffic to AWS resources. DNS converts human-readable domain names into IP addresses.

Using Route 53, domain names can be linked to EC2 instances, load balancers, or S3-hosted websites. DNS records such as A records and CNAME records are configured to route traffic correctly. Route 53 improves application availability through health checks and traffic routing policies. Using domain names instead of IP addresses makes applications easier to access and more professional for users.